The Banksys

Build an AI art-learning assistant in Python + Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

What you're building 3

Setup 4

The big picture 5

Build the features 6

Feature 1: Learn about art styles 6

Feature 2: Choose an improvement area 6

Feature 3: Build the AI prompt 6

Optional extensions 8

What you're building

The Banksys is a web app that helps you learn to draw better. You can read about art styles, upload your own artwork, pick something to improve (like shading or color), and get personalized feedback from an AI art teacher.

The app already launches. You write the three pieces that make it smart:

1. Look up and show a description of an art style.
2. Turn an improvement area into instructions for the AI.
3. Build the prompt that tells the AI how to critique your artwork.

Setup

Open a terminal in the `the_banksys` folder (on Windows, open the folder in File Explorer, type `powershell` in the address bar and press Enter). Then run the commands for your computer, one at a time.

Mac:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

Windows:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python app.py
```

Open the address it prints (usually `http://127.0.0.1:7860`). Do the `activate` line again every time you open a new terminal. **After you edit a Python file, stop the app with `Ctrl + C` and run `python app.py` again.**

> **NOTE**
> **No API key? No problem.** Without a key the app runs in **demo mode**: the AI never looks at your picture, and instead shows the prompt your code built. That is enough to finish every feature. For real feedback, create a Gemini key at aistudio.google.com/apikey (or ask your teacher) and put it in `.env` as `GEMINI_API_KEY=your-key`, with no quotes or spaces. Never paste a key into a Python file or share your `.env`.

> **STUCK?**
> On Windows, if `python` is not found try `py`. If PowerShell blocks `activate`, use Command Prompt instead.

The big picture

Every click in the browser follows the same path:

```
Browser (you click)
   |
   v
app.py          Gradio calls a Python function with your inputs
   |
   +--> art_styles.py   style name  -->  description          (Feature 1)
   |
   +--> feedback.py     area        -->  instructions         (Feature 2)
                        style + area + instructions --> prompt (Feature 3)
                              |
                              v
                        ai_helper.py   sends image + prompt to Gemini
                              |
                              v
                        feedback.py    organizes the answer
   |
   v
Browser (you read the result)
```

File	What it does
`app.py`	The web page: tabs, dropdowns, buttons. It calls your functions.
`art_styles.py`	The dictionary of art styles. **You edit this.**
`feedback.py`	Improvement areas, the prompt, and the answer formatting. **You edit this.**
`ai_helper.py`	The only file that talks to the AI. You never need to edit it.

`app.py` and `ai_helper.py` are finished. Gradio passes a dropdown's current value straight into your function as an argument, and shows whatever your function returns.

Build the features

Until you do a feature, its function returns a placeholder, so the app still runs. Do the features in order.

Feature 1: Learn about art styles

File: `art_styles.py`

Goal: When someone picks a style in tab 1, show its description.

Your Task: Finish `get_style_description(style_name)`. It should return the description for the chosen style from the `ART_STYLES` dictionary. It must not crash if nothing is chosen yet, or if the name is not in the dictionary. Return a friendly message in those cases.

Hint: A dictionary looks things up by key. `.get()` lets you give a default for a missing key, and an empty choice counts as `False` in an `if`.

Test: Run the app, open **1. Learn About Styles**, and pick each style. You should see a different description each time.

Feature 2: Choose an improvement area

File: `feedback.py`

Goal: Turn the area the user picked (Shading, Composition, Color, Proportions or General feedback) into one sentence that tells the AI what to focus on.

Your Task: Finish `get_improvement_instructions(area)` so each area in `IMPROVEMENT_AREAS` returns its own sentence. Think about what an art teacher would look at for each one. Anything unexpected should return an instruction that covers the whole artwork.

Hint: Compare `area` to each choice with `if` / `elif`, and use `else` for everything left over. Compare with `==` (two equals signs).

Test: In the terminal, with the venv on, run:

```
python -c "from feedback import get_improvement_instructions as g; print(g('Color'));
    print(g('Shading')); print(g('Banana'))"
```

You should see three different sentences.

Feature 3: Build the AI prompt

File: `feedback.py`

Goal: Write the text that is sent to the AI along with the picture. A prompt is where you control the AI's behavior, so a clear one gives much better feedback.

Your Task: Finish `build_prompt(style, area, instructions)` and return one string. Your prompt should:

- give the AI a role,
- mention the target style and the improvement area,

- include the `instructions` from Feature 2,
- ask for the answer in three sections labelled exactly `STRENGTHS:` , `AREAS TO IMPROVE:` and `EXERCISES:` .

The finished `format_feedback` function splits the AI's answer on those three labels, so spelling them exactly matters. If the AI skips a label, the raw answer is shown instead.

Hint: Build the prompt in pieces with an f-string such as `f"... the {style} style ..."` , or with `+=` , then `return` the whole thing.

Test: Go to tab 3, choose a target style and an area, and click **Get feedback** (upload a picture in tab 2 first).

- **Demo mode:** the app prints your prompt. Read it as if you were the AI. Does it contain the role, the style, the area, the focus sentence and the three labels? Change the style or area and check the prompt changes.
- **With a key:** you get a formatted critique with **Strengths**, **Areas to Improve** and **Suggested Exercises**. Try the same picture with three different areas and compare the advice.

> **STUCK?**
> If you still see a placeholder, you did not save the file or did not restart the app. If the real answer is not split into sections, check the spelling of your three labels.

Optional extensions

If you finish early, try one of these:

- Add three more art styles (Surrealism, Art Nouveau, Ukiyo-e) and a second target style the AI should compare against.
- Add a "skill level" choice (beginner or advanced) that changes how the prompt is worded.
- Make each style a dictionary with a description **and** a list of famous artists, and show both in tab 1.
- Add a tone setting: gentle teacher, strict critic, or friendly coach.
- Keep a history of past critiques in the app so you can compare your progress.